# Smart Study: Adaptive Exam Practice And Interview Preparation Platform

A Report Submitted
in Partial Fulfillment for the Degree of

**Bachelor of Engineering**
in
**[Name of the Branch]**
(Session [20__–20__])


**Submitted By:**
[Student Name 1] ([Roll No.])
[Student Name 2] ([Roll No.])
[Student Name 3] ([Roll No.])

**Guided By:**
[Name of Project Guide]
[Designation]

Department of [Department Name]
Institute of Engineering & Technology
Vikramshila Campus
Devi Ahilya Vishwavidyalaya, Indore (M.P.)
https://ietdavv.edu.in


October 2026

# CERTIFICATE

The project report entitled **"Smart Study: Adaptive Exam Practice And Interview Preparation Platform"** submitted by **[Full Name of Project Team Members]** is approved for partial fulfillment for the award of Bachelor of Engineering in **[Name of the Branch]** degree by Devi Ahilya Vishwavidyalaya, Indore.

Internal Examiner

External Examiner

# RECOMMENDATION

The dissertation entitled **"Smart Study: Adaptive Exam Practice And Interview Preparation Platform"** submitted by **[Full Name of Project Members]** is a satisfactory account of the bona fide work done under my supervision and is recommended towards the partial fulfillment for the award of Bachelor of Engineering in **[Name of the Branch]** degree by Devi Ahilya Vishwavidyalaya, Indore.

Date: [DD/MM/YYYY]

[Signature]
**[Name of Project Guide]**
Department of [Department Name]

[Signature and Seal]
Counter-signature of HOD

# CANDIDATE DECLARATION

We hereby declare that the work presented in this project entitled **"Smart Study: Adaptive Exam Practice And Interview Preparation Platform"**, in partial fulfillment of the degree of Bachelor of Engineering in **[Name of the Branch]**, is an authentic record of work carried out under the supervision and guidance of **[Name and Designation of Project Guide]**, Department of **[Department Name]**, Institute of Engineering and Technology, Devi Ahilya Vishwavidyalaya, Indore.

We are responsible for the material embodied in this project. This project has not been submitted, in whole or in part, for the award of any other degree or diploma. All external sources used in preparing this report are acknowledged in the references.

Date: [DD/MM/YYYY]
Place: Indore

[Signature of Student 1]
[Name of Student 1]

[Signature of Student 2]
[Name of Student 2]

[Signature of Student 3]
[Name of Student 3]

# SELF-DECLARATION OF PLAGIARISM REPORT

The project report entitled **“Smart Study: Adaptive Exam Practice And Interview Preparation Platform”** submitted by **[Full Name of Project Members]** in partial fulfillment of the requirements for the award of Bachelor of Engineering in **[Name of the Branch]** has been checked for plagiarism using **[Name of Institute-approved plagiarism detection tool]**.

The similarity index of the report is **[To be entered after the official plagiarism check]%**, which is within the permissible limit as per institute and university norms.

We declare that the work submitted is original, that materials from other sources have been cited, and that we accept responsibility for any discrepancy identified in the future.

**Signature of Student(s):**

1. _____
2. _____
3. _____

Date: [DD/MM/YYYY]

> The similarity percentage is intentionally left blank. Run the institute-approved check on the final, student-reviewed report and enter its authentic result before submission.

# ACKNOWLEDGEMENTS

We express our sincere gratitude to our project guide, **[Name of Project Guide]**, for the guidance, technical direction and constructive feedback provided throughout the development of Smart Study. The discussions on project scope, software design and evaluation helped us refine the system and this report.

We thank the Head of the Department, **[Name of HOD]**, for the academic support and facilities made available to us. We are also grateful to the Director and the Institute of Engineering and Technology, Devi Ahilya Vishwavidyalaya, for providing the institutional environment in which this work was undertaken.

We acknowledge the maintainers of the open-source frameworks and libraries used in this project, including Spring Boot, FastAPI, PostgreSQL, scikit-learn and Faster-Whisper. Their documentation and software made it possible to implement and test the application. Finally, we thank our families and peers for their encouragement during the project.

**Personalization required:** Replace the bracketed names and edit this page to reflect the actual guidance and resources received by the project team.

# ABSTRACT

Students preparing for examinations often have to locate recurring topics across past papers, create additional practice questions and judge what to study next. Smart Study is a web-based learning application that brings these activities into one workflow. A student organizes work by subject and topic, uploads past examination papers in PDF format, and uses extracted text to identify question themes and recurring questions. The application can generate original multiple-choice or subjective practice questions from a selected topic and available study material. Its quiz service records responses and adjusts an ability estimate so that the next question is selected near the learner's current estimated level. Subjective responses can receive model-assisted scores and feedback. A separate interview-practice workflow supports academic-subject or engineering-role sessions, typed responses and browser-recorded speech transcribed by a local Whisper model in the ML container. The system is implemented as a browser frontend, a Spring Boot API, a Python FastAPI machine-learning service and a PostgreSQL database, deployed together with Docker Compose. This report presents the problem, related learning systems, requirements, design, data model, implementation and test evidence. The current repository includes automated tests for core quiz, analysis, PDF and transcription behavior; their results and the limits of the available evidence are reported explicitly. Smart Study is a practice and revision aid, not a replacement for instructor judgment or an accredited examination platform. Its usefulness depends on the quality of uploaded material and generated feedback, and generated content should be reviewed by learners and instructors.

**Keywords:** adaptive learning, examination preparation, question generation, item response theory, past-paper analysis, interview practice.

# TABLE OF CONTENTS


Certificate i
Recommendation ii
Candidate Declaration iii
Self-Declaration of Plagiarism Report iv
Acknowledgements v
Abstract vi
**Chapter 1 — Introduction 1**
1.1 Overview and Issues Involved 1
1.2 Problem Definition 1
1.3 Proposed Solution 2
**Chapter 2 — Literature Survey 4**
2.1 Existing Solutions 4
2.2 Methodology and Concepts 5
**Chapter 3 — Analysis and Design 8**
3.1 Software Requirements 8
3.2 Hardware Requirements 9
3.3 Analysis Diagrams and Use Cases 10
3.4 Design Diagrams 12
**Chapter 4 — Implementation and Testing 14**
4.1 Database Design 14
4.2 Class Design 15
4.3 Test Cases and Results 16
**Chapter 5 — Conclusion 19**
References 21
Appendix A — User Workflow and API 22
Appendix B — Submission Completion Checklist 23

# CHAPTER 1
# INTRODUCTION

## 1.1 Overview and Issues Involved

Past examination papers are valuable because they expose learners to the wording, topic emphasis and level of questions used in earlier assessments. In practice, however, a student may have to collect papers from several locations, read them manually, notice repeated concepts, prepare a personal question bank and then decide which question to attempt next. These steps are time-consuming and do not automatically provide targeted feedback. A static list of questions also gives every learner the same sequence even when their knowledge differs.

Smart Study addresses this workflow through a browser application organized around subjects and topics. Users can maintain their own subject areas, upload past papers, inspect recurring themes, generate new practice questions, take a quiz that selects questions according to an evolving ability estimate, and rehearse academic or engineering interviews. The system combines deterministic software components (storage, parsing, topic organization, quiz state and ability updates) with machine-learning services for text analysis, question generation, written feedback and speech transcription.

The application is designed as a study aid. Text extraction may be imperfect for scanned or poorly formatted documents; semantic clustering can group questions that share vocabulary without being conceptually identical; and generated questions or scores may contain errors. Consequently, the application should present its outputs as guidance and preserve the learner's responsibility to verify educational content.

## 1.2 Problem Definition

University learners lack a single, account-based workflow that converts their own past-paper material into organized, recurring-topic insights and adaptive practice while also supporting written-answer feedback and interview rehearsal. Manual analysis is repetitive; static practice does not adapt to demonstrated performance; and separate tools often divide content management, practice and feedback across different interfaces.

**Project objective.** Design and implement a deployable web system that securely organizes study content, extracts and analyzes uploaded examination papers, generates topic-linked practice, adapts quiz difficulty from responses, and supports conversational interview practice with optional voice input.

## Project objectives

- Provide account registration, login and user-scoped access to subjects and study material.

- Allow learners to create subjects/topics and upload PDF question papers with an optional examination year.
- Extract text from digital PDFs and route scanned documents to a protected OCR service.
- Group extracted questions and report recurring questions and topic clusters across papers.
- Generate original multiple-choice and subjective questions and conduct adaptive quizzes.
- Provide rubric-guided feedback for subjective answers and conduct subject/role interview sessions.
- Support browser audio capture and local-service transcription without sending interview audio to an external speech provider.

### Scope and constraints

The implemented scope is a single-page browser interface, Spring Boot REST API, FastAPI ML service and PostgreSQL-backed storage, with Docker Compose as the local deployment route. The ML text-generation integration uses an Ollama-compatible endpoint configured through environment variables. Speech recognition uses Faster-Whisper in the ML container. A working provider key and configured services are required for model-backed features. No claim is made that the model outputs are guaranteed correct, that the system is a high-stakes assessment tool, or that the current repository represents a production-scale hosted service.

## 1.3 Proposed Solution

Smart Study uses a service-oriented design. The frontend presents sign-in, subject dashboards, paper management, practice and interview views. The backend authenticates requests, checks ownership, persists domain data and coordinates calls to the ML service. The ML service exposes typed HTTP endpoints for analysis, question generation, grading, interview turns, scanned-PDF OCR and audio transcription. PostgreSQL stores structured entities; uploaded PDFs are kept in a persistent uploads volume. The system is composed with Docker so that its service boundaries and dependencies can be started together.

The paper-analysis pipeline extracts question-like text, normalizes vocabulary, creates TF-IDF features and clusters questions using K-means. Candidate cluster counts are compared using the silhouette score within a configured bound. Similar questions across different papers are joined using cosine similarity and a configurable threshold; repeated concepts are returned with occurrence and year metadata. This produces an inspection aid rather than a guarantee of syllabus coverage.

Adaptive practice maintains an ability estimate for a quiz session. The implementation uses a logistic response probability and a bounded update after each answer, then selects a remaining question whose difficulty is closest to the current estimate. This is a simplified item-response-

inspired procedure; it is not a calibrated psychometric instrument and does not estimate all parameters of a full item response model.

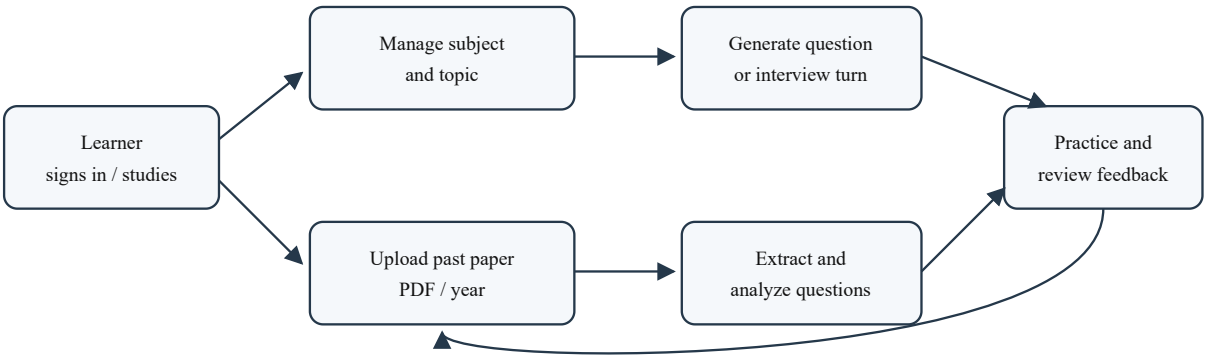



Figure 1.1: High-level study workflow. Generated and analyzed material returns to a learner-controlled practice and review loop.

## Expected contribution

The project's contribution is the integration of paper organization, recurring-question analysis, question generation, adaptive practice and interview rehearsal in one subject-centered application. Compared with static question lists, the quiz loop responds to prior answers; compared with standalone model prompts, the workflow stores user-owned topics, papers, questions and quiz sessions in a structured application. These are design-level advantages; comparative learning outcomes have not been established by a controlled educational study.

# CHAPTER 2
# LITERATURE SURVEY

## 2.1 Existing Solutions

Digital learning products commonly provide some combination of structured lessons, flashcards, assessment authoring, question banks and progress tracking. The following review compares representative tools and categories relevant to Smart Study. It is a functional comparison based on publicly described product capabilities, not a benchmark or independent evaluation of learning effectiveness.

### 2.1.1 Learning management systems and Moodle Quiz

Moodle is an open-source learning management system used to deliver course content, activities and assessments. Its Quiz activity supports instructor-authored questions, attempts, grading and feedback within a course [6]. It is a capable platform when an institution configures the course and question bank. Smart Study differs in its focus on an individual learner's past examination papers, automatic cross-paper topic/repetition analysis and adaptive practice workflow. Moodle's extensive administration model may be more than a learner needs for personal revision.

### 2.1.2 Quizlet

Quizlet centers on user- or teacher-created study sets and offers digital flashcard and practice experiences [7]. Its strength is quick vocabulary and concept review supported by a large content ecosystem. A user typically needs prepared study material; it does not constitute the project's specific pipeline for parsing a learner's examination-paper collection, clustering paper questions and using those patterns to seed topic-linked adaptive quiz sessions.

### 2.1.3 Khan Academy

Khan Academy provides structured lessons and practice across supported subjects, with learner progress linked to a curated curriculum [8]. Its strength is the combination of instructional material and exercises. Smart Study is instead designed to start from a learner's own examination papers and topic organization. It does not attempt to reproduce Khan Academy's broad curriculum, instructional video library or institution-scale learning analytics.

### 2.1.4 Commercial exam-preparation platforms

Exam-preparation services commonly provide syllabus-aligned courses, mock examinations, question banks and performance reports for particular examinations. Their curated content can provide consistency and exam-specific guidance. Their scope is often tied to a provider's content catalog and target assessment. Smart Study's intended distinction is learner-supplied paper analysis and customizable subjects rather than a centrally curated, examination-specific question bank.

### 2.1.5 AI study assistants and question generators

General-purpose AI assistants and educational question generators can create explanations, practice prompts and conversational feedback. They offer flexible interaction but depend on the supplied context and the model's reliability. A standalone chat does not automatically provide a persistent relation between a learner's subjects, uploaded papers, generated question records, quiz state and past-paper recurrence. Smart Study wraps model services in a structured subject-and-quiz workflow and validates generated question fields before storing them.

### 2.1.6 Speech-enabled interview practice

Interview simulators and voice assistants can help candidates rehearse responses. Smart Study's interview workflow supports subject or engineering-role targets, question sequences and feedback; the browser can capture audio and submit it to the local ML service for transcription. This is a focused practice workflow, not a hiring assessment, professional interview credential or guarantee of interview performance.

### Research concepts informing the project

Item response theory models the probability of a response in relation to learner ability and item properties. The project borrows the intuition that practice can be selected near an evolving estimate of ability, while implementing a deliberately simpler bounded logistic update [1, 2]. TF-IDF assigns greater importance to terms that are informative within a document collection, while K-means groups vector representations around centroids [3, 4]. The silhouette coefficient can help compare candidate clusterings by considering compactness and separation [5]. These techniques are used as practical analysis components; none alone understands educational semantics or proves that a cluster is pedagogically coherent.

**Gap and justification.** The reviewed systems typically emphasize curated courses, flashcard sets, instructor-managed assessments or open-ended generative chat. The project addresses a narrower integration gap: a learner can bring their own paper corpus, organize it by subject, inspect recurring patterns and continue directly into generated and adaptive practice, with interview rehearsal in the same application. The gap is functional and architectural; user studies are still required to demonstrate learning benefit.

## 2.2 Methodology and Conceptual Requirements

### 2.2.1 Development methodology

The project follows an iterative implementation approach. First, the repository's user-facing workflow and domain objects are defined. Next, API contracts and persistence entities are implemented in the Spring Boot service. The Python service supplies analysis and model-backed tasks behind typed request/response schemas. The browser interface consumes the API, while Docker Compose connects the application and database. Automated unit, integration and API tests are used to check important behaviors. Iterations address the paper-import path, adaptive

quiz lifecycle, interview turn flow and voice transcription. Configuration values and secrets are supplied through environment variables rather than committed source.

### 2.2.2 Concepts and algorithms

- **REST service:** An HTTP API exposes resources and actions using JSON or multipart form data. The frontend communicates with the backend; the backend calls the internal ML service.
- **PDF extraction and OCR:** PDFBox extracts embedded text from digitally generated documents. When scanned pages contain no usable text, an authenticated internal endpoint can render pages and run OCR. Extraction quality depends on document clarity and language support.
- **TF-IDF:** Each question is represented by weighted one- and two-word terms. Frequent general examination words are removed to reduce uninformative similarity.
- **K-means and silhouette selection:** K-means groups the sparse text vectors. The analysis chooses a cluster count by checking silhouette scores for a bounded set of candidates; deterministic random seeds make the process more reproducible.
- **Cosine similarity and repeated questions:** Questions above a similarity threshold are linked only across different paper IDs. Connected groups supply the repeated-question list and years.
- **Adaptive quiz:** A logistic probability estimates the chance of a correct response given ability and difficulty. After an answer, the estimate is updated by a diminishing step and clamped to a supported range. The next eligible item is the one closest to the current estimate.
- **Structured generation and feedback:** The ML service requests schema-constrained JSON and validates question type, option count, correct-answer index and reference-answer requirements. Subjective feedback is model-assisted and should be treated as formative feedback.
- **Speech transcription:** The browser records an audio response and sends it to the internal API. Faster-Whisper performs local CPU transcription in the ML container; the resulting text is submitted as the candidate response.

### 2.2.3 Security and privacy considerations

Authenticated application routes use JWT-based identity and user ownership checks. Internal OCR/transcription calls are protected by a shared service key and constant-time comparison. Deployment secrets, database credentials and model-provider keys are configured externally. Uploaded papers and audio contain potentially sensitive information, so operators should restrict access to volumes, use HTTPS outside localhost, establish retention rules, and avoid uploading material without the right to use it. Model providers may process text sent for generation or

grading; operators must review provider terms and avoid sensitive personal data. The local transcription path is designed not to send captured interview audio to an external speech provider.

# CHAPTER 3
# ANALYSIS AND DESIGN

## 3.1 Software Requirements

### 3.1.1 Functional requirements

| ID | Requirement | Acceptance description |
|---|---|---|
| FR-01 | Account access | A visitor can register or sign in; protected actions are associated with the authenticated user. |
| FR-02 | Subject and topic management | A learner can create subjects and topics and view their own study areas. |
| FR-03 | Paper management | A learner can upload a PDF to a subject, optionally record its exam year, list papers and delete a paper. |
| FR-04 | Paper analysis | Text from uploaded papers is analyzed into question clusters and cross-paper recurring-question results. |
| FR-05 | Question generation | The system can request multiple-choice or subjective questions for selected topics and persist usable results. |
| FR-06 | Adaptive quiz | The learner can answer quiz questions, receive results/feedback and progress until completion. |
| FR-07 | Subjective assessment | A written response is evaluated against a reference answer and rubric, returning a bounded score and feedback. |
| FR-08 | Interview practice | A learner can choose an academic subject or engineering role, answer a question sequence and receive feedback. |
| FR-09 | Voice response | A learner can record a response in the browser and submit audio for transcription by the local ML service. |

### 3.1.2 Non-functional requirements

- **Security:** User-owned data must be checked against the authenticated principal; internal ML endpoints must not be exposed as public application routes.
- **Reliability:** Invalid PDF, malformed model responses and unavailable dependencies must produce explicit errors rather than fabricated success results.
- **Usability:** Primary workflows must be available in a modern browser, with typed answers retained as an alternative to voice.
- **Maintainability:** Frontend, API, ML service and persistence responsibilities are separated and communicate through explicit contracts.
- **Portability:** Dockerfiles and Docker Compose describe the local multi-service deployment; secrets and runtime configuration remain external.

- **Performance:** File-size and question-count limits bound expensive operations. Exact throughput and latency targets have not yet been established by load testing.
- **Data integrity:** Relational ownership and foreign-key associations keep each subject, paper, topic and quiz connected to its user context.

### 3.1.3 Technologies and tools

| Component | Technology | Reason for selection |
|---|---|---|
| User interface | HTML, CSS, JavaScript, Nginx | Lightweight browser client; static assets are served separately from API processing. |
| Application API | Java 25, Spring Boot, Spring Web MVC, Spring Data JPA, Spring Security | Typed REST endpoints, persistence integration and authentication support. |
| Persistence | PostgreSQL 16 | Relational storage for users, subjects, papers, questions and quiz sessions. |
| ML service | Python, FastAPI, Pydantic, scikit-learn | Typed service contracts and established text-vectorization and clustering implementations. |
| PDF | Apache PDFBox; pdf2image and OCR dependencies for scans | Extract embedded PDF text and provide a separate OCR path for scanned documents. |
| Language model | Ollama-compatible API | Question generation, subjective feedback and interview interaction are configured independently of application code. |
| Speech | Faster-Whisper | Local-container transcription so audio does not require an external speech provider. |
| Deployment | Docker, Docker Compose | Build and connect the frontend, backend, ML service and database consistently. |
| Testing | JUnit / Spring Boot Test; pytest | Unit, integration and API-level checks for Java and Python components. |

## 3.2 Hardware Requirements

**Development/deployment host (minimum practical baseline):** 64-bit Windows, macOS or Linux capable of running Docker Desktop/Engine; a multi-core CPU; 8 GB RAM for a small local trial (16 GB or more recommended when PostgreSQL, the Java service and the local speech model run together); several GB of free disk for images, uploaded papers and the downloaded Whisper model; and a microphone for voice practice. The exact memory and disk requirement depends on the chosen Whisper model, OCR packages and concurrent workload.

**Client device:** A recent laptop, desktop or mobile device with a standards-compliant browser and network access to the application. Microphone access requires a secure context such as localhost

or HTTPS and explicit browser permission. If the client microphone is unavailable, interview answers can be typed.

**External services:** The configured Ollama endpoint and model must be reachable for question generation, grading and conversational interview responses. When using a hosted provider, a valid API key is required. These are operational dependencies, not hardware requirements of the learner's browser.

## 3.3 Analysis Diagrams

### 3.3.1 Use Case Model

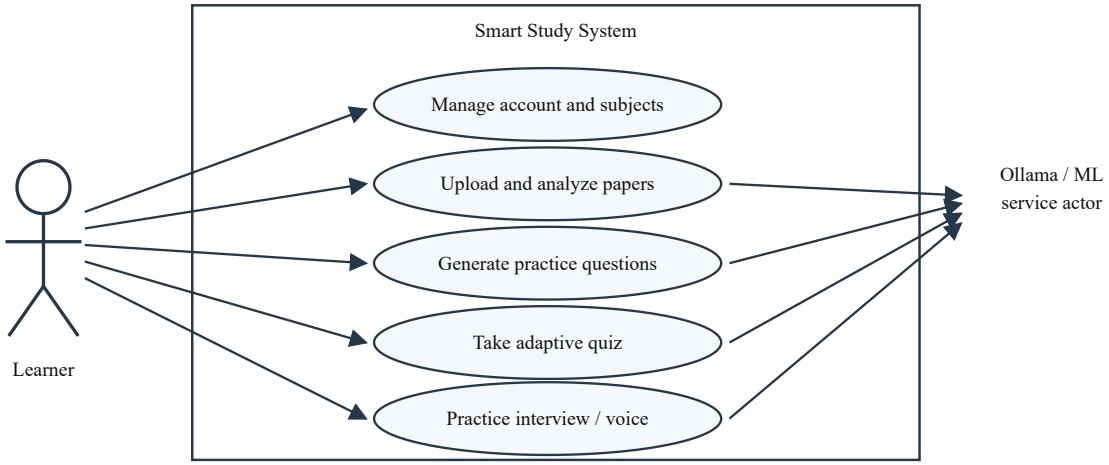



Figure 3.1: Primary external actor and supporting ML-service interactions.

The learner initiates all primary use cases. The backend mediates access to persistent data and calls the ML service only for operations that need analysis, generation, grading, interview responses or transcription. The provider is therefore a supporting service, not an independent user.

### 3.3.2 Use Case Descriptions (Rational Unified Process Style)

**UC-01 — Register or Sign In**

**Primary actor:** Learner. **Preconditions:** Application is reachable; learner has a valid email and password for the selected action. **Trigger:** Learner submits the registration or login form. **Main flow:** (1) System validates required fields. (2) For registration, the service creates the account if the email is not already in use; for login, it verifies credentials. (3) On success, the service issues an authentication token and the client opens the subject dashboard. **Alternatives:** Invalid fields, duplicate email or rejected credentials return an error and leave the user unauthenticated. **Postconditions:** Successful authentication establishes the user context for subsequent requests.

**UC-02 — Create Subject and Topic**

**Primary actor:** Authenticated learner. **Preconditions:** Valid authenticated session. **Trigger:** Learner submits a subject or topic form. **Main flow:** (1) Client sends the name and related parent identifier. (2) Backend validates the request and confirms parent ownership. (3) Backend stores the new record and returns it to the client. **Alternatives:** Missing/invalid names or a resource owned by another user are rejected. **Postconditions:** The subject/topic is available within that learner's workspace.

**UC-03 — Upload and Analyze Past Papers**

**Primary actor:** Authenticated learner. **Preconditions:** Learner owns a subject; input is a supported PDF within configured upload limits. **Trigger:** Learner uploads a paper and optionally supplies its year. **Main flow:** (1) Backend verifies subject ownership and stores the file under a generated name. (2) Text is extracted from the PDF; if the normal extraction path yields no usable text, the configured OCR flow may be used. (3) Paper metadata and text are associated with the subject. (4) On analysis request, extracted questions are normalized, vectorized, clustered and compared across papers. (5) Cluster and recurrence results are returned. **Alternatives:** Invalid PDF, unreadable scan, no papers or unavailable ML service produce an explicit error. Failed upload processing removes the newly stored file where possible. **Postconditions:** A successful upload is attached to its subject; analysis is returned without changing the source paper.

**UC-04 — Generate Practice Questions**

**Primary actor:** Authenticated learner. **Preconditions:** Learner owns a subject and has selected one or more topics. **Trigger:** Learner requests a practice set. **Main flow:** (1) Backend composes a typed generation request with subject, topics, requested question type and optional paper context. (2) ML service requests structured output from the configured language model. (3) Response fields and question-specific rules are validated. (4) Usable questions are stored against their topic and returned. **Alternatives:** Provider unavailable, malformed output or no valid questions are reported as failure; invalid model items are not silently presented as questions. **Postconditions:** Valid questions are available for later practice.

**UC-05 — Complete an Adaptive Quiz**

**Primary actor:** Authenticated learner. **Preconditions:** Learner owns the subject; a usable question pool exists. **Trigger:** Learner starts a quiz. **Main flow:** (1) Backend creates a quiz session and selects the initial question. (2) Learner submits an answer. (3) Backend records correctness, updates ability and answer counts, and chooses the next question closest to current ability. (4) System returns answer feedback and progress. (5) Session ends at its configured maximum or when no questions remain. **Alternatives:** An invalid session/question/answer is rejected; subjective answers are submitted to the grading service and may be unavailable if that dependency is down. **Postconditions:** Completed session records its status, counts and final estimate.

**UC-06 — Practice an Interview**

**Primary actor:** Authenticated learner. **Preconditions:** Interview service and, for voice, browser microphone permission are available. **Trigger:** Learner chooses a subject or engineering role, level and question count. **Main flow:** (1) Client requests the next interview turn with prior conversation context. (2) ML service returns a question. (3) Learner types or records an answer. (4) Recorded audio is sent through the protected transcription route; transcription text becomes the answer. (5) Service evaluates the answer and returns feedback/score or the next question. (6) Session completes at the selected limit. **Alternatives:** Microphone denial or transcription failure leaves text entry available; service errors are displayed. **Postconditions:** Learner receives session feedback; browser audio is not sent to an external speech provider by this design.

## 3.4 Design Diagrams

### 3.4.1 Architecture Diagram and System Overview

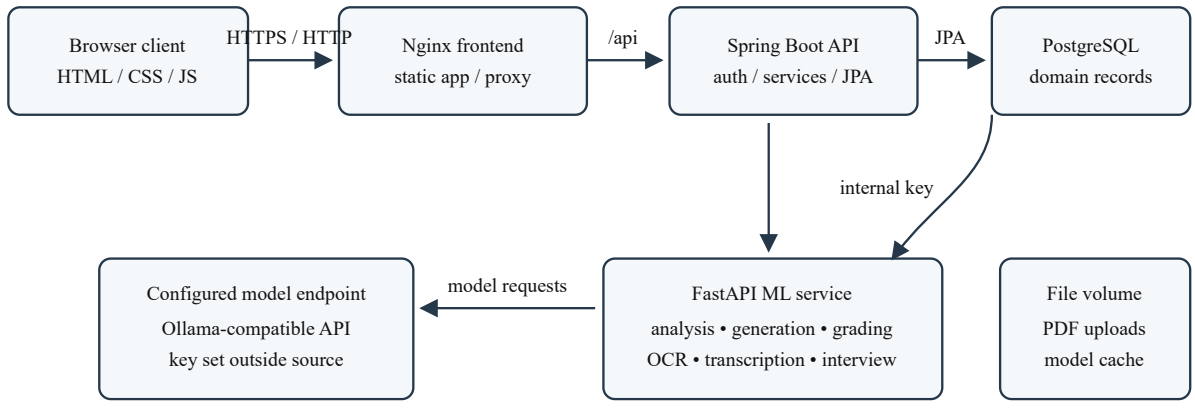



Figure 3.2: Logical deployment architecture. Docker Compose connects the frontend, API, ML service and database; persistent volumes hold uploaded documents and the transcription model cache.

The browser interacts with the static application and API. Nginx serves frontend assets and routes API traffic according to the deployment configuration. The Spring Boot service owns user-facing authentication, authorization, domain operations and database access. It calls FastAPI using service-to-service configuration for machine-learning tasks. FastAPI performs text clustering and calls the configured language-model endpoint for generation and conversational tasks. PostgreSQL persists relational data, while file storage and the Whisper cache use persistent Docker volumes. Each container is configured through environment variables; sensitive values must not be committed.

### 3.4.2 Sequence Diagram: Paper Upload and Analysis

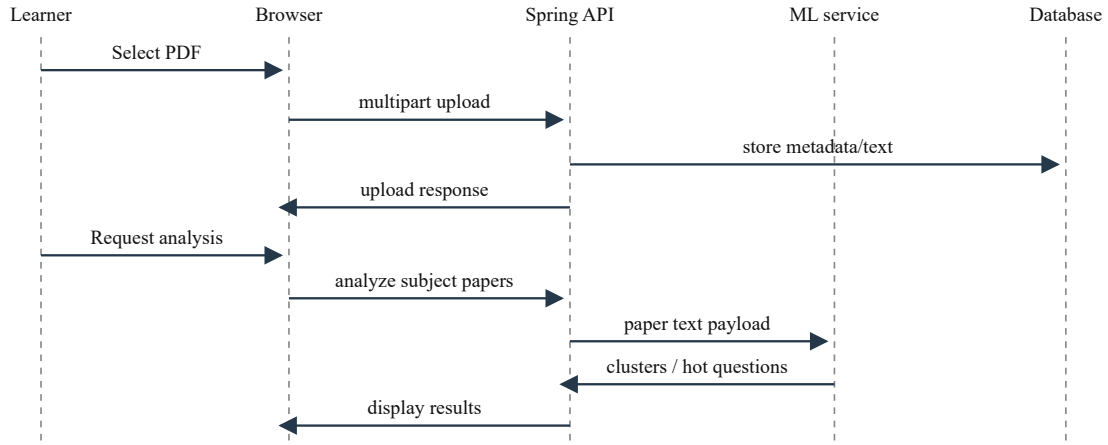



Figure 3.3: Sequence for paper upload followed by analysis; the ML service response is mediated by the backend.

# CHAPTER 4
# IMPLEMENTATION AND TESTING

## 4.1 Database Design

The backend uses JPA entities mapped to relational tables. The following ER view summarizes the core relations implemented in the domain model. A learner owns subjects; subjects contain topics and uploaded papers; topics contain questions; quiz sessions refer to a learner and subject and maintain a question pool; answers link a session to the question answered. Question options are persisted as an ordered element collection. The many-to-many question pool is represented by a join table.

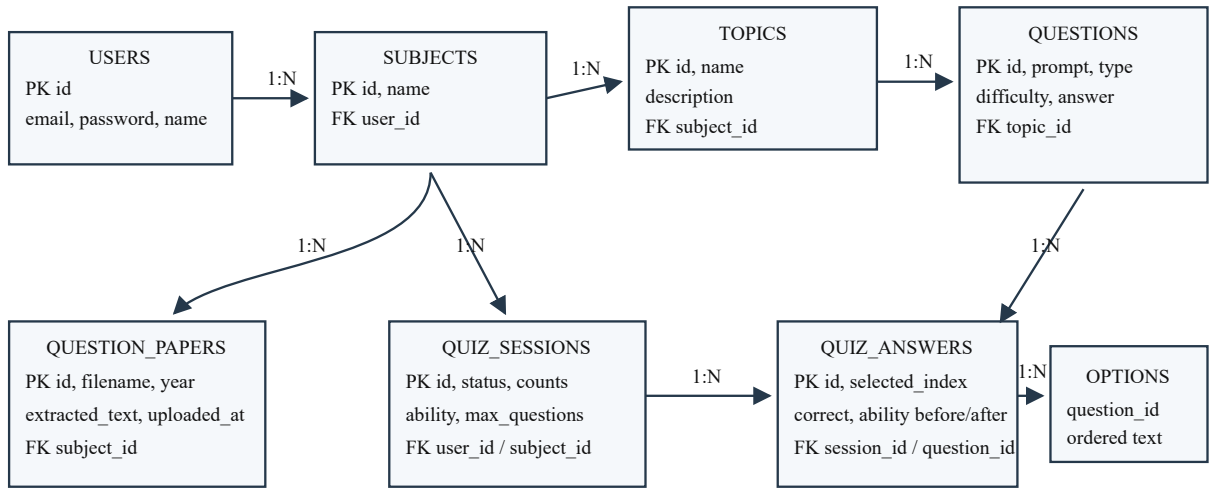



Figure 4.1: Core entity-relationship model. PK denotes primary key; FK denotes foreign key; 1:N denotes one-to-many.

| Table | Representative fields | Purpose |
|---|---|---|
| users | id (PK), email (unique, required), bcrypt-hashed password, full_name (required) | Account identity and credential record; the service hashes passwords with BCrypt and verifies them through the configured password encoder. |
| subjects | id (PK), name, user_id (FK) | Top-level learner-owned study area. |
| topics | id (PK), name, description, subject_id (FK) | Topic organization within a subject. |
| question_papers | id (PK), original_filename, stored_filename (unique), exam_year, uploaded_at, extracted_text, subject_id (FK) | Paper metadata and extracted text; binary file is stored outside the relational row. |

| | | |
|---|---|---|
| questions | id (PK), prompt, correct_index, question_type, reference_answer, explanation, difficulty, times_answered, topic_id (FK) | Generated or curated question content and adaptive difficulty metadata. |
| question_options | question_id (FK), option_order, option_text | Ordered answer options persisted as an element collection. |
| quiz_sessions | id (PK), user_id (FK), subject_id (FK), max_questions, answered_count, correct_count, ability_estimate, status, current_question_id (FK) | State and progress for an adaptive quiz. |
| quiz_session_questions | session_id (FK), question_id (FK), question_order | Ordered question pool associated with a quiz session. |
| quiz_answers | id (PK), session_id (FK), question_id (FK), selected_index, correct, ability_before, ability_after | Answer-level record supporting session progress and audit of ability changes. |

The schema is created/updated by the JPA configuration in the current project. Before production deployment, schema changes should be managed with explicit versioned migrations and retention/backup policies. The database model does not store interview conversation history as a persistent entity in the core domain model shown here.

## 4.2 Class Design and Explanation

The diagram below shows the central domain and service classes rather than every framework DTO, repository and configuration class in the codebase. It reflects the project's main responsibility boundaries.

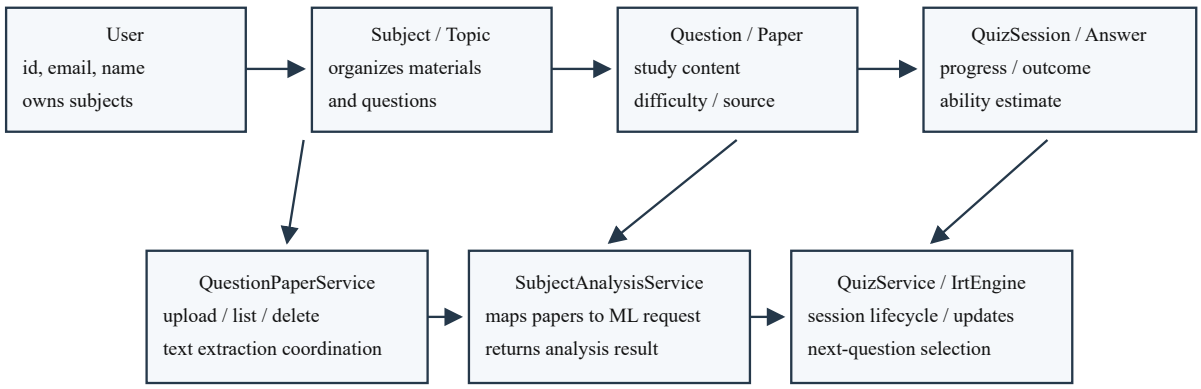



Figure 4.2: Core domain classes and coordinating services.

| Class / component | Purpose and principal behavior |
|---|---|
| User | JPA entity for account identity. Holds ID, email, credential value and full name; forms the ownership root for user-specific subjects and quiz sessions. |

| | |
|---|---|
| Subject | JPA entity representing a learner’s study area. Associates to an owner and contains topics. |
| Topic | JPA entity representing a named topic and optional description within a subject. |
| QuestionPaper | JPA entity holding source filename, stored filename, exam year, upload timestamp, extracted text and owning subject. |
| Question | JPA entity for a practice prompt, ordered options, answer metadata, question type, explanation, difficulty and topic. It increments the number of answers and exposes question-type-aware accessors. |
| QuizSession | JPA entity that records learner, subject, maximum question count, progress, ability estimate, current question, status and question pool. `recordAnswer` updates counters/ability; `finish` closes the session. |
| QuizAnswer | JPA entity linking an answer to its session and question, retaining selected option, correctness and ability before/after the response. |
| IrtEngine | Stateless utility. `probabilityCorrect(ability, difficulty)` evaluates a logistic response probability; `information` returns p(1-p); `updateAbility` and `updateDifficulty` apply bounded diminishing-step updates; `selectNext` selects the candidate closest to estimated ability. |
| QuestionPaperService | Coordinates ownership validation, generated filename storage, PDF text extraction, persistence, listing, re-extraction when text is missing and deletion. It cleans up a newly stored file when upload processing fails. |
| SubjectAnalysisService | Loads a user’s subject papers, verifies readable text, maps data to the ML request and returns timestamped analysis metadata. |
| ML service (`analyzer`, `llm_client`) | Python modules split and normalize questions, build TF-IDF features, cluster and identify recurring text. The language-model client generates typed questions, grades subjective answers and manages interview turns. |

## 4.3 Test Cases and Results

### 4.3.1 Testing approach

Unit tests cover deterministic algorithms and service-level rules; integration tests exercise multi-step API and quiz flows; Python tests cover the ML service’s analysis, request handling, model-client behavior, OCR and transcription. Manual browser testing should verify layout, validation, authentication, upload feedback, quiz transitions, microphone permission and responsive behavior. The test evidence below distinguishes tests executed in this report-preparation environment from results already recorded in the repository.

### 4.3.2 Automated test evidence available

| Suite / evidence | Test count | Recorded outcome | Evidence note |
|---|---|---|---|

| | | | |
|---|---|---|---|
| ML service pytest suite | 35 | 35 passed | Executed during report preparation on 25 October 2026. |
| Backend IRT engine | 14 | 14 passed | Existing Maven Surefire report in the repository. |
| Backend quiz flow integration | 10 | 10 passed | Existing Maven Surefire report in the repository. |
| Backend quiz service | 10 | 10 passed | Existing Maven Surefire report in the repository. |
| Backend application context | 1 | 1 passed | Existing Maven Surefire report in the repository. |
| Backend PDF text extractor | 1 | 1 passed | Existing Maven Surefire report in the repository. |
| Backend question paper service | 1 | 1 passed | Existing Maven Surefire report in the repository. |

These recorded backend reports total 37 passing tests and contain no recorded failures or errors. A fresh Maven test invocation during report preparation could not start because the Java runtime failed a native-memory allocation. Therefore, the 37 backend passes refer to the checked-in workspace's existing Surefire evidence and are not represented as a fresh run. Browser end-to-end screenshots and external-provider live tests were not captured for this report.

### 4.3.3 Representative functional test cases

| ID | Scenario and input | Expected result | Observed evidence/status |
|---|---|---|---|
| TC-01 | Register with valid unique email, name and password. | Account is created and authenticated user reaches the subject dashboard. | Automated backend application test exists; re-check with deployment credentials. |
| TC-02 | Sign in with invalid credentials. | Authentication is rejected and an error is displayed; no protected data is returned. | Manual UI verification required. |
| TC-03 | Create a subject and topic using non-empty names. | Records are persisted and appear in the learner's subject workflow. | Manual UI verification required. |
| TC-04 | Upload a valid text-based PDF to an owned subject with a year. | Paper metadata and extracted text are stored; file is listed for the subject. | PDF extractor and paper-service backend tests are recorded as passing. |
| TC-05 | Upload malformed/unsupported PDF or a scan with no legible text. | Action fails with a useful validation message and does not leave a successful-looking record. | PDF/OCR Python tests pass; verify with representative documents before submission. |

| | | | |
|---|---|---|---|
| TC-06 | Analyze a subject with papers containing similar questions across different years. | Question clusters and repeated-question records include source paper/year metadata. | ML analyzer tests are included in the 35 passing pytest tests. |
| TC-07 | Request multiple-choice generation for a topic. | Each accepted item has four options, one valid correct index and topic metadata. | ML generation contract tests are included in the 35 passing pytest tests; live provider not exercised here. |
| TC-08 | Answer correct and incorrect questions at different estimated abilities. | Ability is bounded and updated; next question is selected from the remaining pool near estimated ability. | IRT (14) and quiz flow (10) recorded tests pass. |
| TC-09 | Submit a subjective response of at least the configured minimum length. | Response receives a normalized score and constructive feedback; provider errors are surfaced. | Live model-provider result not verified in this test run. |
| TC-10 | Complete a subject/engineering interview with typed answers. | Turns advance, feedback is returned, and the session ends at the configured count. | API/unit coverage exists; verify against a configured model endpoint. |
| TC-11 | Deny microphone permission, then submit a typed response. | Voice capture is unavailable but typed answer remains usable. | Manual browser verification required. |
| TC-12 | Attempt to access another user's subject/paper. | Backend rejects cross-user access and does not disclose the resource. | Ownership checks exist in service flow; add/execute explicit authorization tests before production. |

> **Submission evidence still required:** Capture real screenshots of registration/login, subject dashboard, paper upload/analysis, quiz feedback and interview setup/session in the configured application. Add each image with a caption, test input, expected result and observed result. No mock screen is presented as a runtime screenshot in this draft.

# CHAPTER 5
# CONCLUSION

## 5.1 Conclusion

Smart Study demonstrates an integrated web application for organizing examination preparation around learner-supplied subject material. Its implemented workflow connects account-scoped subjects and topics, PDF paper ingestion, question and recurrence analysis, generated practice, adaptive quiz sessions, subjective feedback and interview rehearsal. A Java API coordinates persistent domain operations, while a Python ML service isolates text analysis, language-model interaction, OCR and speech transcription. Docker Compose describes a repeatable local multi-service deployment.

The project also demonstrates how a simple ability estimate can influence practice selection. The implemented logistic update and nearest-difficulty selection make quiz sessions responsive to learner answers, while remaining intentionally simpler than a fully calibrated item-response assessment. Automated Python tests executed for this report passed 35 tests. Existing backend test reports record 37 passing tests across six suites, but the fresh backend test command could not run in the available environment because of JVM native-memory allocation failure. These results support important implementation paths; they do not establish educational effectiveness, model accuracy, accessibility compliance, load capacity or production readiness.

The main practical value is workflow integration rather than a claim that automated generation or clustering is inherently superior to instructor-authored learning. Uploaded paper quality, correct user configuration, reliable model responses, review of generated items and secure operations remain necessary. Before institutional submission or broader use, the team should complete real-browser screenshots, verify the full deployment with its approved provider configuration, add explicit authorization and failure-path tests, inspect model output quality with educators, and update this document with authentic student, guide and plagiarism-check details.

## 5.2 Limitations

- TF-IDF and lexical similarity may miss paraphrases or create false matches; cluster labels are descriptive terms, not validated syllabus topics.
- Question generation and subjective grading depend on an external language model and can be incorrect, inconsistent or biased.
- The adaptive algorithm is a simplified pedagogical heuristic; it has not been calibrated on a representative learner/item dataset.
- OCR accuracy varies with scan quality, layout and language; extraction should be inspected before relying on results.

- No controlled learner study, formal accessibility review, production load test or availability evaluation is included in the current evidence.
- The current local deployment requires operator-provided secrets, database setup and model-service configuration.

## 5.3 Future Scope

- Conduct educator and learner evaluation; measure task completion, usefulness, question quality and learning outcomes.
- Add instructor review/approval workflows and citations to the uploaded source passages behind generated questions.
- Improve semantic matching with embedding-based retrieval, while retaining transparent thresholds and human review.
- Evaluate calibrated IRT models using sufficient response data, item calibration, uncertainty estimates and privacy-preserving analytics.
- Add versioned database migrations, retention controls, backups, accessibility testing, security assessment and operational monitoring.
- Support more document types and languages after measuring extraction accuracy and defining data-handling requirements.

# REFERENCES


1. F. M. Lord, M. R. Novick and A. Birnbaum, *Statistical Theories of Mental Test Scores*, Addison-Wesley, 1968.
2. F. B. Baker and S.-H. Kim, *Item Response Theory: Parameter Estimation Techniques*, 2nd ed., Marcel Dekker, 2004.
3. G. Salton and M. J. McGill, *Introduction to Modern Information Retrieval*, McGraw-Hill, 1983.
4. J. MacQueen, "Some Methods for Classification and Analysis of Multivariate Observations," in *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, pp. 281–297, 1967.
5. P. J. Rousseeuw, "Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis," *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987.
6. Moodle Documentation, "Quiz activity." Available at: https://docs.moodle.org/en/Quiz_activity. Accessed 25 October 2026.
7. Quizlet, "Study with flashcards and learning tools." Available at: https://quizlet.com/. Accessed 25 October 2026.
8. Khan Academy, "Practice." Available at: https://www.khanacademy.org/. Accessed 25 October 2026.
9. Spring, "Spring Boot Reference Documentation." Available at: https://docs.spring.io/spring-boot/reference/. Accessed 25 October 2026.
10. FastAPI, "FastAPI Documentation." Available at: https://fastapi.tiangolo.com/. Accessed 25 October 2026.
11. PostgreSQL Global Development Group, "PostgreSQL Documentation." Available at: https://www.postgresql.org/docs/. Accessed 25 October 2026.
12. scikit-learn developers, "Clustering" and "Feature extraction." Available at: https://scikit-learn.org/stable/. Accessed 25 October 2026.
13. Ollama, "Ollama API documentation." Available at: https://docs.ollama.com/. Accessed 25 October 2026.
14. SYSTRAN, "Faster-Whisper: Faster Whisper transcription with CTranslate2." Available at: https://github.com/SYSTRAN/faster-whisper. Accessed 25 October 2026.
15. Project source repository, *Smart Study*: README, backend source, frontend source, ML service source, and automated tests. Repository workspace reviewed on 25 October 2026.

# APPENDIX A — USER WORKFLOW AND API SUMMARY

## A.1 Suggested demonstration workflow

1. Start the services using the Docker Compose instructions in the project README and provide locally generated secrets and model-provider configuration.
2. Register a demonstration account and create a subject and topic.
3. Upload a legally usable sample question-paper PDF and verify its extracted text and year.
4. Run analysis and inspect clusters and recurring questions against the original paper.
5. Generate a practice set; review generated questions for correctness before attempting them.
6. Take a short quiz and observe feedback and question progression.
7. Run a typed interview session. If using voice, grant microphone permission and verify transcription before submitting the answer.
8. Capture actual application screenshots for the final printed report, avoiding real credentials and personal data.

## A.2 Selected API surface

| Route family | Purpose |
|---|---|
| `/api/auth/register`, `/api/auth/login` | Account registration and login. |
| `/api/subjects`, `/api/subjects/{id}/topics` | Subject and topic organization. |
| `/api/subjects/{id}/papers`, `/api/papers/{id}/text` | Paper upload, listing, extracted text and deletion. |
| `/api/subjects/{id}/analysis` | Past-paper question analysis. |
| `/api/subjects/{id}/questions/generate-from-papers` | Generate questions using subject paper material. |
| `/api/topics/{id}/questions/generate` | Generate questions for a topic. |
| `/api/subjects/{id}/quiz`, `/api/quiz/{sessionId}/answer` | Start and answer a quiz; subjective-answer route is provided separately. |
| `/api/interview/turn`, `/api/interview/transcribe` | Interview question/feedback turn and authenticated transcription relay. |

Exact request schemas, authorization rules and error responses are defined by the current backend controllers, DTOs and ML service schemas; consult source/API documentation before integrating another client.

# APPENDIX B — SUBMISSION COMPLETION CHECKLIST

| Item | Action before binding/submission | Status in this draft |
|---|---|---|
| Project title | Confirm title wording and capitalization with department. | Drafted from repository purpose. |
| Student names and roll numbers | Replace all bracketed team-member fields on cover, declaration and plagiarism page. | Required from team. |
| Branch, department and session | Enter official branch, department name and academic session. | Required from team. |
| Guide and HOD | Enter official names/designations; obtain signatures and seal as required. | Required from institute/team. |
| Plagiarism result | Run the approved institutional tool on the final report and enter its actual similarity percentage. | Intentionally blank. |
| Application screenshots | Capture actual screens and add user input, expected output and observed result for each. | Not captured; do not substitute illustrative diagrams. |
| Table of contents pages | Refresh chapter page references after screenshots and final pagination. | Chapter references intentionally left blank. |
| References | Verify source details/URLs and add any additional material actually consulted. | Draft references included. |
| Final technical verification | Run backend and ML tests in the project's intended environment; record exact command, date and outcomes. | ML suite passed; backend fresh run blocked by host JVM memory. |
| Print format | Use Times New Roman, justified text, 1.5 line spacing, required paper and color prints per institute instruction. | Print-ready HTML/PDF uses Times New Roman, A4, and 1.5 body line spacing. |

> **Draft status:** This report is based on the project source and README available in the workspace. It is not a substitute for team review, guide approval, verified screenshots, institutional plagiarism checking or correction of bracketed details.